



Chapter 3 – Agile Software Development

Topics covered



- ✧ **Agile methods**
- ✧ **Agile development techniques**
- ✧ **Agile project management**
- ✧ **Scaling agile methods**

Rapid software development



✧ Plan-driven development –

- Not **essential for some types of system.**

✧ **Rapid development** and delivery is now often the most important requirement for software systems

- Businesses operate in a fast –changing requirement
- Software has to evolve quickly to reflect changing business needs.

Agile methods

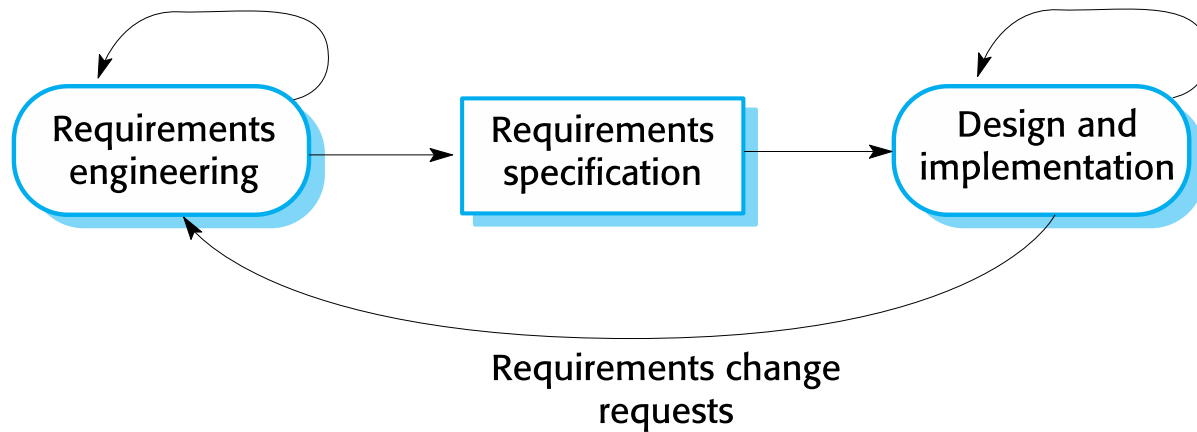
✧ Agile development methods

- aim to radically reduce the delivery time

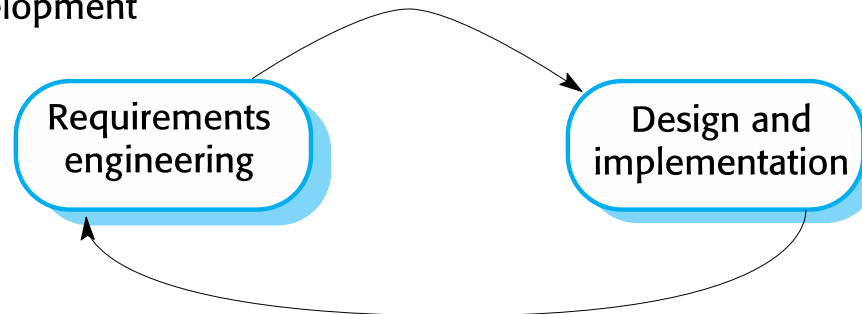
Plan-driven and agile development



Plan-based development



Agile development



Agile methods



Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

✧ **Agile methods** are incremental development methods that focus on rapid software development, frequent releases of the software, reducing process overheads by minimizing documentation and producing high-quality code

- Dissatisfaction with the **overheads** involved in software process (e.g. **documentation**)

The principles of agile methods



Principle	Description
Customer involvement	Customers should be closely involved throughout the development process . Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment.
People not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change and so design the system to accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process . Wherever possible, actively work to eliminate complexity from the system.

Agile method applicability



- ✧ **Product development** - a **small or medium-sized product** for sale.
 - **Virtually all software products and apps are now developed using an agile approach**

- ✧ **Custom system development** - **clear commitment from the customer to become involved** in the development process

- ✧ **Few external rules and regulations** that affect the software.



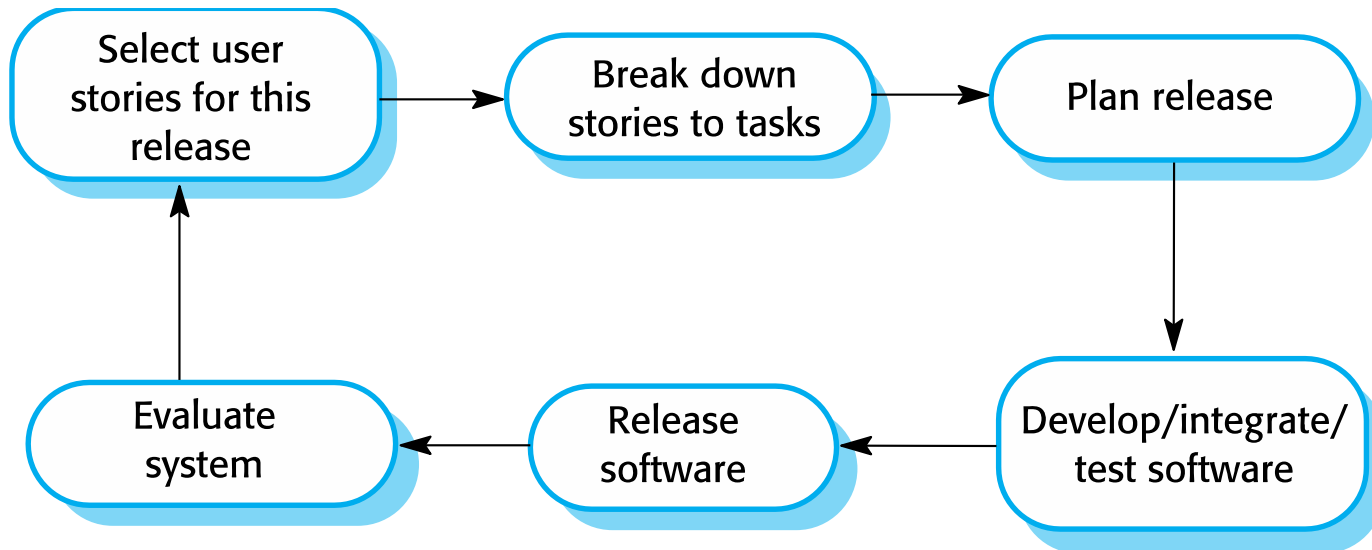
Agile development techniques

Extreme programming



- ✧ Extreme Programming (XP) takes an **'extreme'** approach to iterative development.
 - **Increments** are delivered to customers every **2 weeks**;
 - All **tests** must be run test for every version.

The extreme programming release cycle



Extreme programming practices (a)



Principle or practice	Description
Incremental planning	Requirements are recorded on “ story cards ” and the stories to be included in a release are determined by the time available and their relative priority . The developers break these stories into development ‘Tasks’..
Small releases	The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the first release.
Simple design	Enough design is carried out to meet the current requirements and no more.
Test-first development	An automated unit test framework is used to write tests for a new piece of functionality before that functionality itself is implemented.
Refactoring	All developers are expected to refactor the code continuously as soon as possible code improvements are found. This keeps the code simple and maintainable.

Extreme programming practices (b)



Pair programming	Developers work in pairs, checking each other's work and providing the support to always do a good job .
Collective ownership	The pairs of developers work on all areas of the system, so that no islands of expertise develop and all the developers take responsibility for all of the code . Anyone can change anything.
Continuous integration	As soon as the work on a task is complete, it is integrated into the whole system . After any such integration, all the unit tests in the system must pass .
Sustainable pace	Large amounts of overtime are not considered acceptable as the net effect is often to reduce code quality and medium term productivity
On-site customer	A representative of the end-user of the system (the customer) should be available full time for the use of the XP team. In an extreme programming process, the customer is a member of the development team and is responsible for bringing system requirements to the team for implementation.

Key practices



✧ Key practices:

- User stories for specification
- Refactoring
- Test-first development
- Pair programming

User stories for requirements



- ✧ In XP, a **customer or user** - part of the XP team
- ✧ User requirements are expressed as user **stories or scenarios**.
- ✧ These are **written on cards** and the development team break them down into **implementation tasks**. These tasks are the basis of **schedule** and **cost estimates**.
- ✧ The customer chooses the stories for inclusion in the next release based on their **priorities** and the schedule estimates.



A 'prescribing medication' story

Kate is a doctor who wishes to prescribe medication for a patient attending a clinic. The patient record is already displayed on her computer so she clicks on the medication field and can select 'current medication', 'new medication' or 'formulary'.

If she selects 'current medication', the system asks her to check the dose; If she wants to change the dose, she enters the new dose then confirms the prescription.

If she chooses 'new medication', the system assumes that she knows which medication to prescribe. She types the first few letters of the drug name. The system displays a list of possible drugs starting with these letters. She chooses the required medication and the system responds by asking her to check that the medication selected is correct. She enters the dose then confirms the prescription.

If she chooses 'formulary', the system displays a search box for the approved formulary. She can then search for the drug required. She selects a drug and is asked to check that the medication is correct. She enters the dose then confirms the prescription.

The system always checks that the dose is within the approved range. If it isn't, Kate is asked to change the dose.

After Kate has confirmed the prescription, it will be displayed for checking. She either clicks 'OK' or 'Change'. If she clicks 'OK', the prescription is recorded on the audit database. If she clicks on 'Change', she reenters the 'Prescribing medication' process.

Refactoring



- ✧ XP - proposes **constant code improvement (refactoring)** to make changes easier when they have to be implemented.
 - this **reduces costs later** in the life cycle
 - This **improves the understandability** of the software
 - Changes are easier to make because the **code is well-structured and clear**

- However, some changes requires **architecture refactoring** and this is much more expensive.

Examples of refactoring



- ✧ Re-organization of a class hierarchy to **remove duplicate code**.
- ✧ **Tidying up and renaming attributes and methods** to make them easier to understand.
- ✧ The **replacement of inline code** with calls to methods that have been included in a program library.

Test-first development



- ✧ XP has developed an approach where the **program is tested after every change has been made.**
- ✧ Writing tests before code **clarifies the requirements to be implemented.**
- ✧ Usually relies on a testing framework such as **Junit.**

Pair programming



- ✧ Pair programming involves **programmers working in pairs**, developing code together.
- ✧ This helps develop **common ownership of code** and **spreads knowledge across** the team.
- ✧ It serves as an **informal review process** as each line of code is looked at by more than 1 person.
- ✧ It **encourages refactoring** as the whole team can benefit from improving the system code.

Pair programming



- ✧ Programmers develop the software together.
- ✧ Pairs are created dynamically during the development process.
- ✧ Sharing of knowledge that happens during pair programming
 - reduces the overall risks to a project when team members leave.
- ✧ Pair programming is not necessarily inefficient
 - There is some evidence that suggests that a pair working together is more efficient than 2 programmers working separately.



✧ Scenario:

- ✧ Imagine a software development team that has decided to implement pair programming, where two programmers work together on the same task. As the team evaluates the effectiveness of this approach, they notice that the productivity rate of these pairs is significantly higher than that of two programmers working independently. What could be four potential reasons for this increased productivity when programmers collaborate in pairs?



✧ Scenario:

- ✧ A software development team at TechSolutions Inc. is transitioning from a plan-driven process to Agile methodology to enhance their responsiveness to client feedback and improve project delivery timelines. The team is currently working on a customer relationship management (CRM) system. Describe how the Extreme Programming (XP) process would use its framework activity to change the team's approach to developing the CRM system.



Agile project management

Agile project management



- ✧ Agile project management requires a different approach, which is adapted to:
- ✧ incremental development and
- ✧ practices used in agile methods.

Scrum



- ✧ **Scrum** is an agile method that provides a project management framework and **focuses on managing iterative development** rather than specific agile practices.
- ✧ There are **three phases** in Scrum.
 - The initial phase is **an outline planning phase** where you establish the general objectives for the project and design the software architecture.
 - This is followed by **a series of sprint cycles**, where each cycle develops an increment of the system.
 - The **project closure phase** wraps up the project, completes required documentation such as system help frames and user manuals and assesses the lessons learned from the project.

Scrum terminology (a)



Scrum term	Definition
Development team	A self-organizing group of software developers, which should be no more than 7 people. They are responsible for developing the software and other essential project documents.
Potentially shippable product increment	The software increment that is delivered from a sprint. The idea is that this should be 'potentially shippable' which means that it is in a finished state and no further work, such as testing, is needed to incorporate it into the final product. In practice, this is not always achievable.
Product backlog	This is a list of 'to do' items which the Scrum team must tackle. They may be feature definitions for the software, software requirements, user stories or descriptions of supplementary tasks that are needed, such as architecture definition or user documentation.
Product owner	An individual (or possibly a small group) whose job is to identify product features or requirements, prioritize these for development and continuously review the product backlog to ensure that the project continues to meet critical business needs. The Product Owner can be a customer but might also be a product manager in a software company or other stakeholder representative.

Scrum terminology (b)



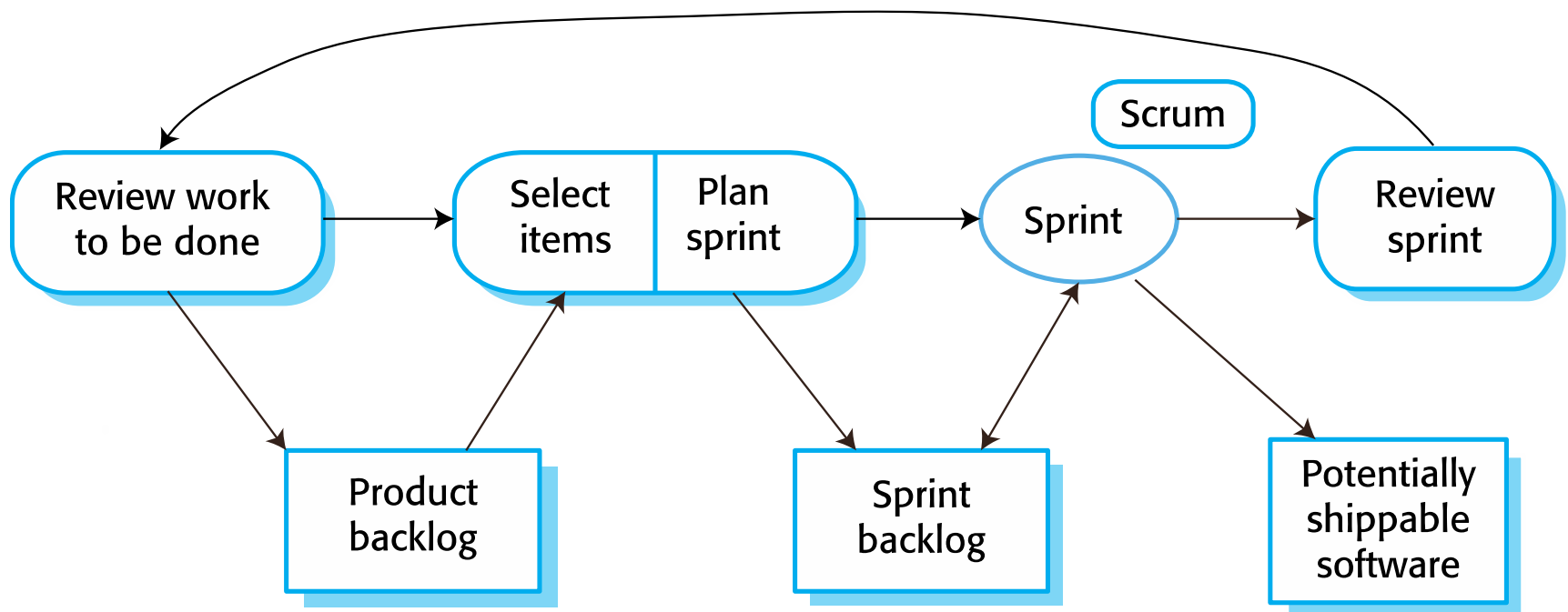
Scrum term	Definition
Scrum	A daily meeting of the Scrum team that reviews progress and prioritizes work to be done that day. Ideally, this should be a short face-to-face meeting that includes the whole team.
ScrumMaster	The ScrumMaster is responsible for ensuring that the Scrum process is followed and guides the team in the effective use of Scrum. He or she is responsible for interfacing with the rest of the company and for ensuring that the Scrum team is not diverted by outside interference. The Scrum developers are adamant that the ScrumMaster should not be thought of as a project manager. Others, however, may not always find it easy to see the difference.
Sprint	A development iteration. Sprints are usually 2-4 weeks long.
Velocity	An estimate of how much product backlog effort that a team can cover in a single sprint. Understanding a team's velocity helps them estimate what can be covered in a sprint and provides a basis for measuring improving performance.

The Scrum sprint cycle



- ✧ Sprints are fixed length, normally 2–4 weeks.
- ✧ The **starting point** for planning is the product backlog.
- ✧ The team organize themselves to develop the software.
- ✧ Meetings and all communications channelled through the so-called '**Scrum master**'.
- ✧ At the **end of the sprint**, the work done is reviewed and presented to stakeholders.
- ✧ The next sprint cycle then begins!

Scrum sprint cycle

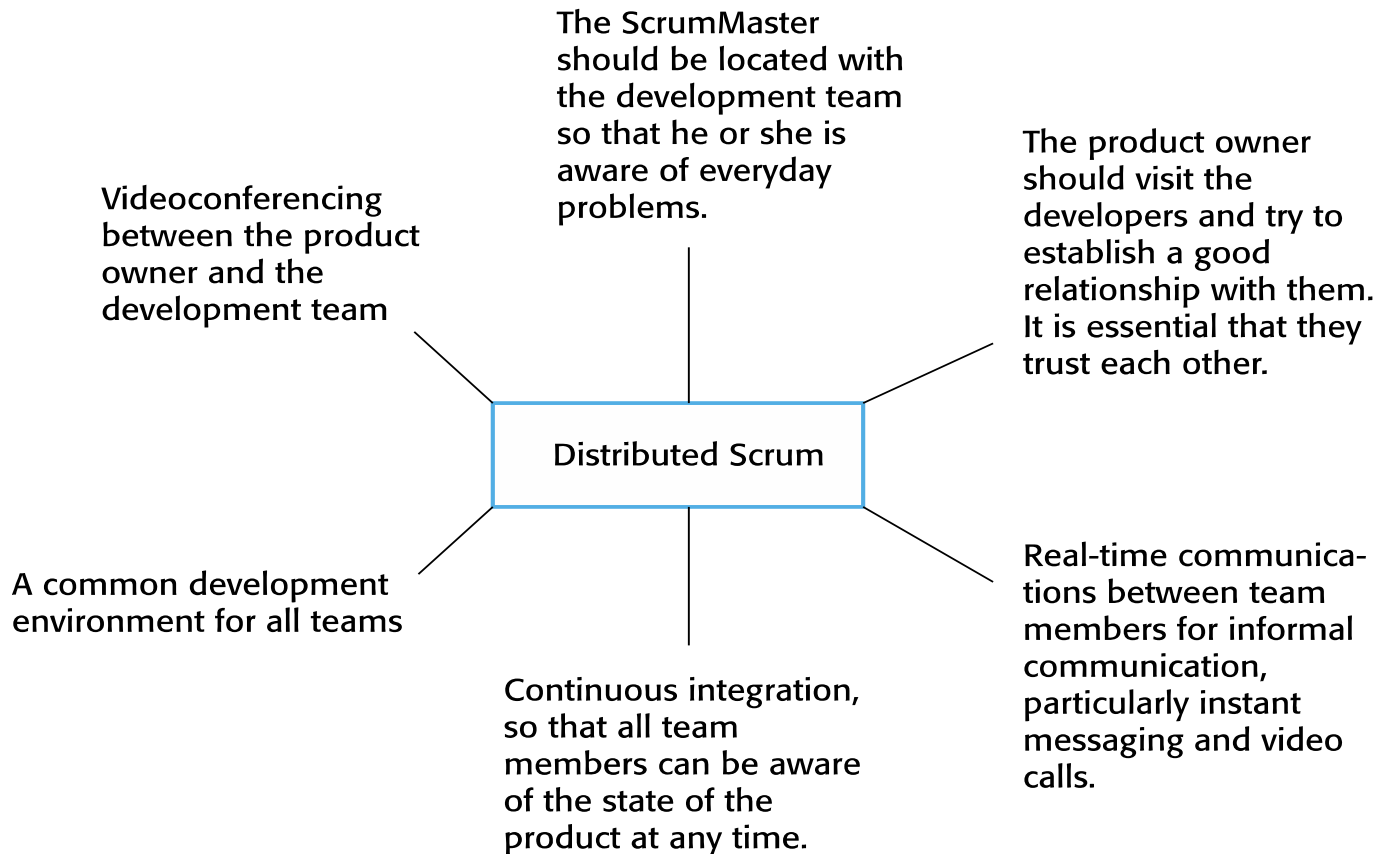


Scrum benefits



- ✧ The product is broken down into a set of manageable and understandable chunks.
- ✧ Unstable requirements do not hold up progress.
- ✧ The whole team have visibility of everything and consequently team communication is improved.
- ✧ Customers see on-time delivery of increments and gain feedback on how the product works.
- ✧ Trust between customers and developers is established.

Distributed Scrum





Scaling agile methods

Scaling agile methods



- ✧ Agile methods - successful for **small and medium sized projects**
- ✧ Success because of improved communications.
- ✧ Scaling up agile methods - cope with larger, longer projects, perhaps working in different locations.
- ✧ Scaling while maintain agile fundamentals:
 - Flexible planning, frequent system releases, continuous integration, test-driven development and good team communications.

Practical problems with agile methods



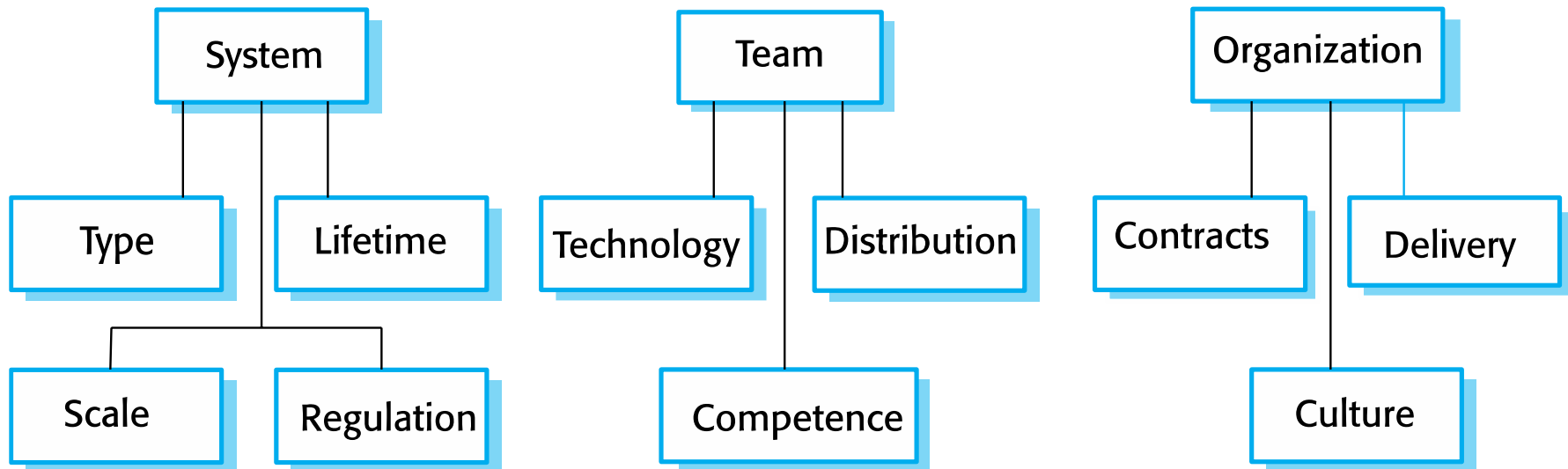
- ✧ Incompatible with the legal approach to contract.
- ✧ Not good for software maintenance.
- ✧ Agile methods are designed for small co-located teams

Agile and plan-driven methods



- ✧ Most projects include elements of plan-driven and agile processes. Deciding on the balance depends on:
- Is it important to have **a very detailed specification and design** before moving to implementation? If so, you probably need to use **a plan-driven approach**.
 - Is an incremental delivery strategy, where you deliver the software to customers and get rapid feedback from them, realistic? If so, consider using **agile methods**.
 - **How large** is the system that is being developed? Agile methods are most effective when the system can be developed with a small co-located team who can communicate informally. This may not be possible for large systems that require larger development teams so a plan-driven approach may have to be used.

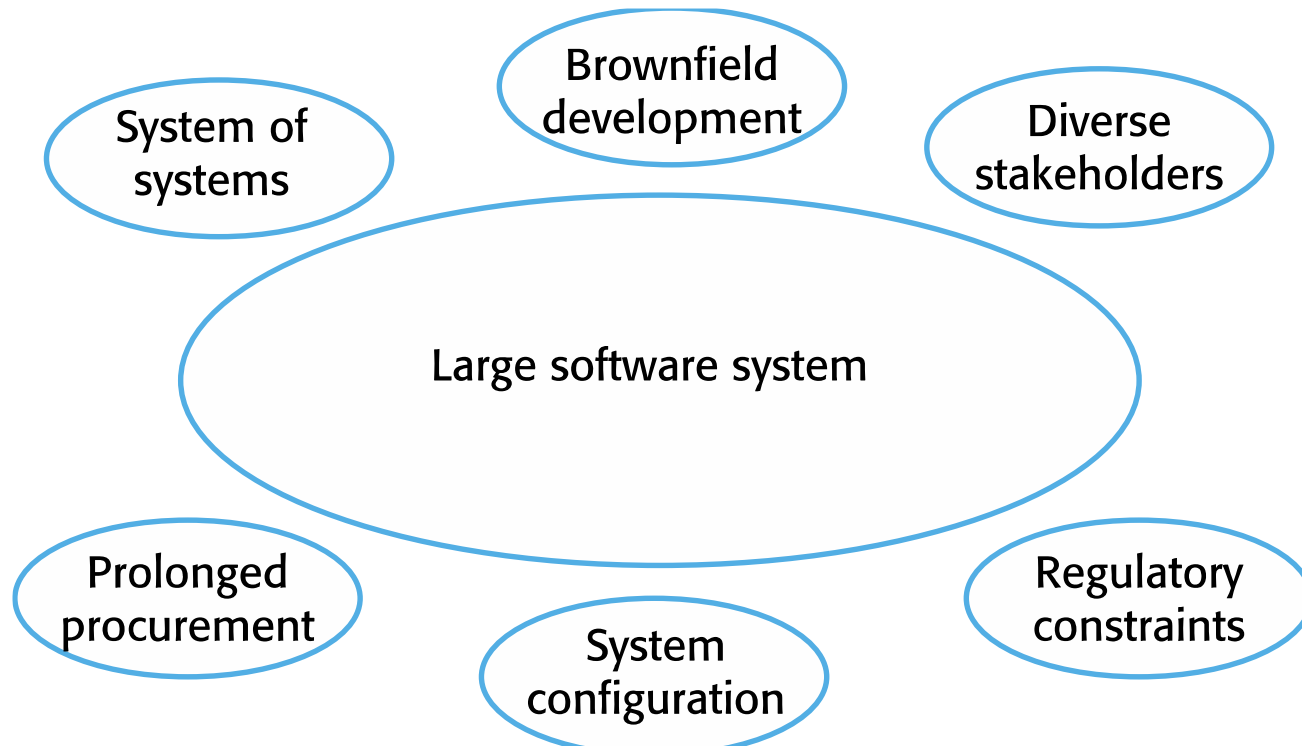
Agile and plan-based factors



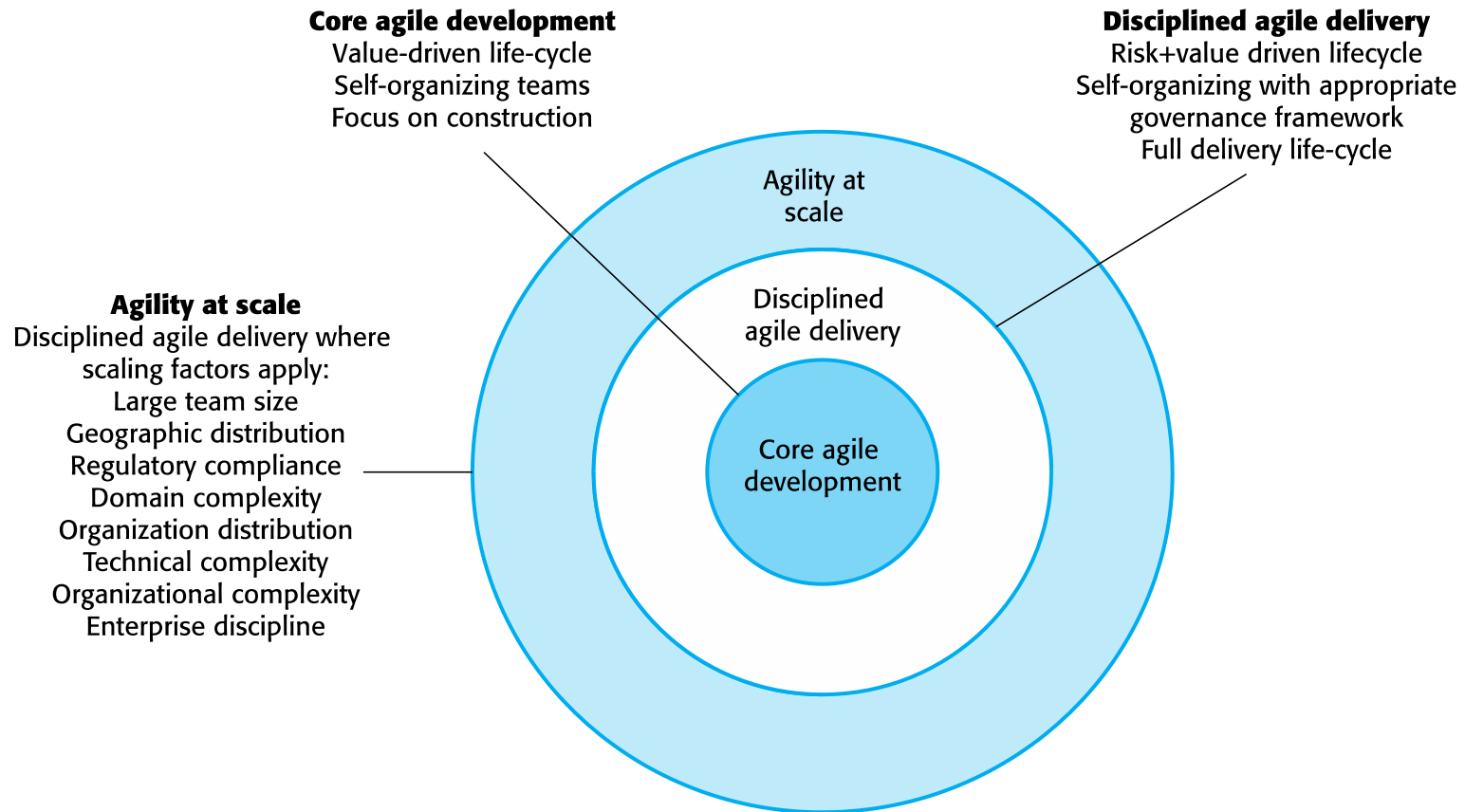
Factors in large systems



- ✧ Large systems are 'brownfield systems', - include and interact with a number of existing systems.
 - Many of the system requirements are concerned with this interaction and so **don't really lend themselves to flexibility and incremental development**



IBM's agility at scale model – LTD!



Issues with Scaling up to large systems



- ✧ A completely incremental approach to requirements engineering is impossible.
- ✧ There cannot be a single product owner or customer representative.
- ✧ For large systems development, it is not possible to focus only on the code of the system.
- ✧ Cross-team communication mechanisms have to be designed and used.
- ✧ Continuous integration is practically impossible. However, it is essential to maintain frequent system builds and regular releases of the system.

Multi-team Scrum



✧ *Role replication*

- Each team has a Product Owner for their work component and ScrumMaster.

✧ *Product architects*

- Each team chooses a product architect and these architects collaborate to design and evolve the overall system architecture.

✧ *Release alignment*

- The dates of product releases from each team are aligned so that a demonstrable and complete system is produced.

✧ *Scrum of Scrums*

- There is a daily Scrum of Scrums where representatives from each team meet to discuss progress and plan work to be done.

Issues that may hinder Agile methods adoption in organizations



- ✧ Project managers who do not have **experience of agile methods** may be reluctant to accept the risk of a new approach.
- ✧ Large organizations often have **quality procedures and standards** that all projects are expected to follow and, because of their bureaucratic nature, these are likely to be incompatible with agile methods.
- ✧ Agile methods seem to work best when team members have a relatively high skill level. However, within large organizations, there are likely to be a **wide range of skills and abilities**.
- ✧ There may be **cultural resistance to agile methods**, especially in those organizations that have a long history of using conventional systems engineering processes.

Question:



- ✧ Your company is developing a new e-commerce platform, and the team consists of 10 co-located developers working closely together.
- ✧ Would an agile approach be suitable for this project? Why or why not?

Answer: Small Agile Team Development



- ✧ Yes!
- ✧ An agile approach would be suitable because the team is relatively small and co-located, allowing for informal communication, quick decision-making, and iterative development, which are key strengths of agile methods.

Scenario: Development of a Large-Scale Healthcare Management System



- ✧ You are part of a software engineering team assigned to develop a comprehensive healthcare management system for a national health service. The system aims to integrate patient records, appointment scheduling, billing, and reporting functionalities across multiple hospitals and clinics.
- ✧ **Context:**
- ✧ The system is expected to serve thousands of users, including doctors, nurses, administrative staff, and patients.
- ✧ The development team consists of 20 members, including designers, programmers, and testers, divided into several subteams that work both locally and remotely.
- ✧ There are strict regulations governing healthcare data security and patient privacy that the system must adhere to.

Questions:



- ✧ **Question-01:** Considering the size of the healthcare management system and the diverse development team structure, what challenges might arise in implementing agile methodologies? Discuss how these challenges could impact team communication and project success.
- ✧ **Question-02:** Given that the healthcare management system requires extensive analysis before implementation, what design considerations should be prioritized during the requirements engineering phase? How does this differ from an agile approach?
- ✧ **Question-03:** How does the expected long lifetime of the healthcare management system influence the architectural design decisions? Discuss the importance of documentation in this context.

Question:



- ✧ **Question-04:** Analyze the implications of external regulations on the requirements engineering process for the healthcare management system. What types of documentation might be necessary to comply with these regulations?
- ✧ **Question-05:** Given the mixed competence levels within the development team and the traditional engineering culture of the organization, how can the project manager effectively bridge the gap between agile practices and the need for detailed documentation?
- ✧ **Question-06:** Discuss how the diversity of stakeholders in the healthcare management system project can complicate the requirements elicitation process. What strategies can be employed to ensure all stakeholder voices are heard?